

APPLIED MACHINE LEARNING LAB

Experiment 5

(CSEG 1021)

SUBMITTED BY:

ALKA BISHT

SAP ID:590031136

SUBMITTED TO:

Dr. Pavinder Yadav

AssistantProfessor

Open in Colab

In [3]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np
```

In [5]:
```
df = pd.read_csv("indian_engineering_placement_2026.csv")
print(df.sample(10))
print(df.shape)
print(df.info())
print(df.describe())
print(df.isnull().sum())
print(df.duplicated().sum())
```

```
      Student_ID  Age  Gender           State      Branch College_Tier  CGPA  \
8366     ST08367   19  Female          Kerala          IT       Tier-3  7.56
12134    ST12135   23  Female           Delhi         CSE       Tier-2  7.06
9528     ST09529   22    Male      Tamil Nadu       Civil       Tier-2  8.25
258      ST00259   19    Male  Andhra Pradesh         CSE       Tier-2  7.13
6236     ST06237   21    Male   Uttar Pradesh  Mechanical       Tier-3  8.56
11636    ST11637   22  Female     Maharashtra         EEE       Tier-3  6.79
10361    ST10362   21    Male      Tamil Nadu       Civil       Tier-2  8.10
10139    ST10140   19  Female     West Bengal         CSE       Tier-2  7.68
3408     ST03409   21    Male          Kerala       Civil       Tier-2  5.79
11115    ST11116   21    Male       Karnataka    Chemical       Tier-2  8.32

       Attendance_Percentage  DSA_Problems_Solved  Aptitude_Score  ...  \
8366                   73.56                  220              65  ...
12134                  75.00                  228              24  ...
9528                   68.21                  157              31  ...
258                    69.71                  231              36  ...
6236                   78.40                  127              49  ...
11636                  70.09                  162               0  ...
10361                  77.21                  185              41  ...
10139                  80.94                  292              32  ...
3408                   70.61                   48              48  ...
11115                  82.10                  207              48  ...

       GitHub_Contributions  LinkedIn_Activity_Score  Hackathons_Participated  \
8366                    904                     51.0                        4
12134                   708                     29.0                        1
9528                    279                     16.0                        4
258                    1452                     24.0                        3
6236                    235                      4.0                        1
11636                   421                     60.0                        0
10361                   365                      1.0                        4
10139                  1189                     38.0                        4
3408                    189                     73.0                        0
11115                   298                     41.0                        1

       Competitive_Programming_Rating  Study_Hours_Per_Week  \
8366                             1773                    13
12134                            1769                    27
9528                              858                    29
258                              1831                    20
6236                              641                    18
11636                             539                     2
10361                             807                    21
```

```
10139                            1608                    20
3408                              370                    14
11115                             787                    24

       Soft_Skills_Score  Leadership_Experience  Extracurricular_Activities  \
8366                  49                     No                           3
12134                 49                     No                           2
9528                  69                     No                           2
258                   47                     No                           3
6236                  49                     No                           1
11636                 70                     No                           4
10361                 47                     No                           1
10139                 58                    Yes                           1
3408                  44                     No                           0
11115                 59                     No                           0

       Placement_Status  Package_LPA
8366             Placed        12.65
12134            Placed         8.11
9528             Placed         6.01
258              Placed        10.75
6236             Placed         7.13
11636            Placed         3.29
10361        Not Placed         0.00
10139            Placed        10.81
3408         Not Placed         0.00
11115            Placed         4.65

[10 rows x 25 columns]
(15000, 25)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 15000 entries, 0 to 14999
Data columns (total 25 columns):
 #   Column                          Non-Null Count  Dtype
---  ------                          --------------  -----
 0   Student_ID                      15000 non-null  object
 1   Age                             15000 non-null  int64
 2   Gender                          15000 non-null  object
 3   State                           15000 non-null  object
 4   Branch                          15000 non-null  object
 5   College_Tier                    15000 non-null  object
 6   CGPA                            15000 non-null  float64
 7   Attendance_Percentage           15000 non-null  float64
 8   DSA_Problems_Solved             15000 non-null  int64
 9   Aptitude_Score                  15000 non-null  int64
 10  Communication_Score             15000 non-null  int64
 11  Projects_Count                  15000 non-null  int64
 12  Internships_Count               15000 non-null  int64
 13  Certifications_Count            15000 non-null  int64
 14  Open_Source_Contributions       14745 non-null  float64
 15  GitHub_Contributions            15000 non-null  int64
 16  LinkedIn_Activity_Score         14775 non-null  float64
 17  Hackathons_Participated         15000 non-null  int64
 18  Competitive_Programming_Rating  15000 non-null  int64
 19  Study_Hours_Per_Week            15000 non-null  int64
 20  Soft_Skills_Score               15000 non-null  int64
 21  Leadership_Experience           15000 non-null  object
 22  Extracurricular_Activities      15000 non-null  int64
 23  Placement_Status                15000 non-null  object
 24  Package_LPA                     15000 non-null  float64
dtypes: float64(5), int64(13), object(7)
memory usage: 2.9+ MB
None
                Age          CGPA  Attendance_Percentage  DSA_Problems_Solved  \
count  15000.000000  15000.000000           15000.000000         15000.000000
```

```
count  15000.000000  15000.000000  15000.000000  15000.000000
mean      21.153400      7.477432     73.774239    181.760667
std        1.536324      0.870838      7.516522     83.611775
min       18.000000      4.050000     50.000000     12.000000
25%       20.000000      6.890000     68.660000    123.000000
50%       21.000000      7.470000     73.720000    176.000000
75%       22.000000      8.060000     78.810000    225.000000
max       25.000000     10.000000    100.000000   1199.000000

       Aptitude_Score  Communication_Score  Projects_Count  Internships_Count  \
count    15000.000000         15000.000000    15000.000000       15000.000000
mean        39.993400            53.912333        3.514600           1.201267
std         14.400763            15.731021        2.004708           1.131979
min          0.000000             0.000000        0.000000           0.000000
25%         30.000000            43.000000        2.000000           0.000000
50%         40.000000            54.000000        3.000000           1.000000
75%         49.000000            64.000000        5.000000           2.000000
max         99.000000           100.000000       14.000000           5.000000

       Certifications_Count  Open_Source_Contributions  GitHub_Contributions  \
count          15000.000000                14745.000000          15000.000000
mean               3.499200                   82.736792            506.151000
std                2.014978                   63.965529            321.159554
min                0.000000                    8.000000             57.000000
25%                2.000000                   44.000000            291.000000
50%                3.000000                   65.000000            421.000000
75%                5.000000                  100.000000            637.000000
max               15.000000                  968.000000           4891.000000

       LinkedIn_Activity_Score  Hackathons_Participated  \
count             14775.000000             15000.000000
mean                 30.124196                 2.206333
std                  18.463372                 1.567096
min                   0.000000                 0.000000
25%                  16.000000                 1.000000
50%                  29.000000                 2.000000
75%                  43.000000                 3.000000
max                 100.000000                10.000000

       Competitive_Programming_Rating  Study_Hours_Per_Week  \
count                    15000.000000          15000.000000
mean                      1121.632800             17.986733
std                        533.458634              8.395494
min                        126.000000              0.000000
25%                        679.000000             12.000000
50%                        912.000000             18.000000
75%                       1585.000000             24.000000
max                       2500.000000             50.000000

       Soft_Skills_Score  Extracurricular_Activities   Package_LPA
count       15000.000000                15000.000000  15000.000000
mean           52.239600                    2.285933      6.547623
std            11.150848                    1.577236      7.273714
min            15.000000                    0.000000      0.000000
25%            45.000000                    1.000000      0.000000
50%            52.000000                    2.000000      5.940000
75%            60.000000                    3.000000     10.170000
max            93.000000                   10.000000     49.990000
Student_ID                          0
Age                                 0
Gender                              0
State                               0
Branch                              0
College_Tier                        0
CGPA                                0
```

```
Attendance_Percentage              0
DSA_Problems_Solved                0
Aptitude_Score                     0
Communication_Score                0
Projects_Count                     0
Internships_Count                  0
Certifications_Count               0
Open_Source_Contributions        255
GitHub_Contributions               0
LinkedIn_Activity_Score          225
Hackathons_Participated            0
Competitive_Programming_Rating     0
Study_Hours_Per_Week               0
Soft_Skills_Score                  0
Leadership_Experience              0
Extracurricular_Activities         0
Placement_Status                   0
Package_LPA                        0
dtype: int64
0
```

In [6]:
```
df = pd.read_csv("indian_engineering_placement_2026.csv")
median =df["Open_Source_Contributions"].quantile(0.5)
df["Open_Source_Contributions"]= df["Open_Source_Contributions"].fillna(median)

mean =df["LinkedIn_Activity_Score"].mean()
df["LinkedIn_Activity_Score"]= df["LinkedIn_Activity_Score"].fillna(mean)

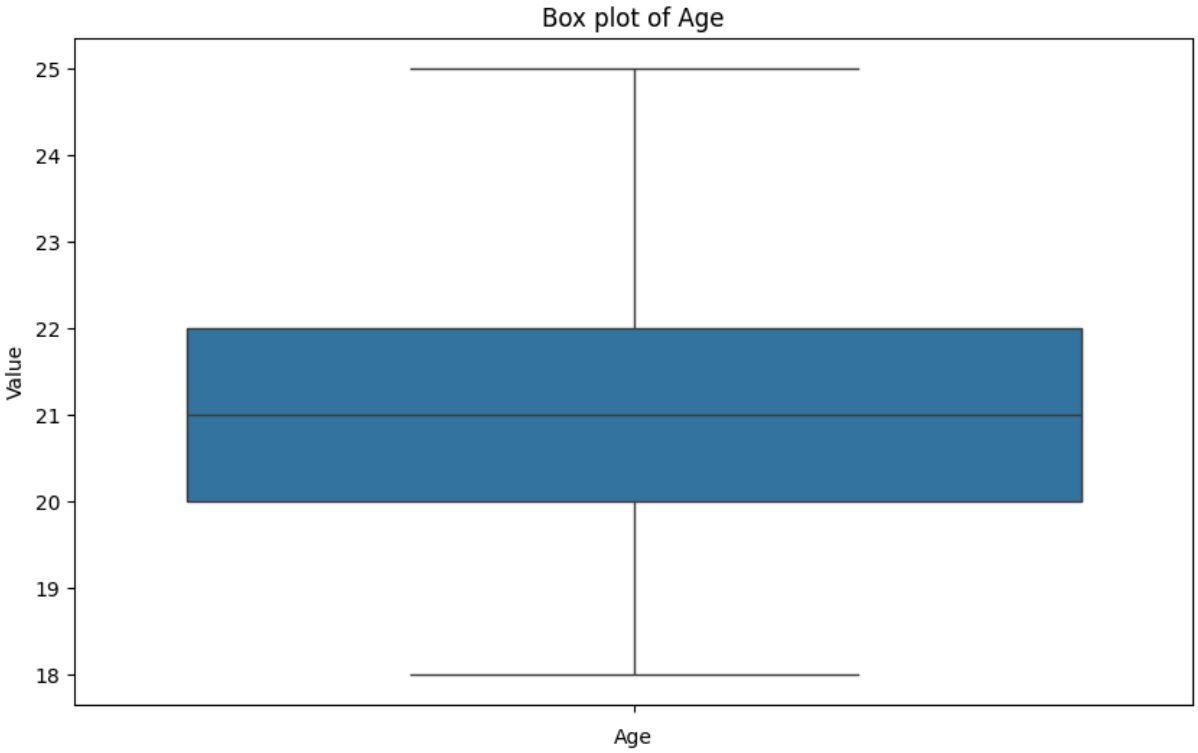
print(df.isnull().sum())
df = df.drop_duplicates()
```

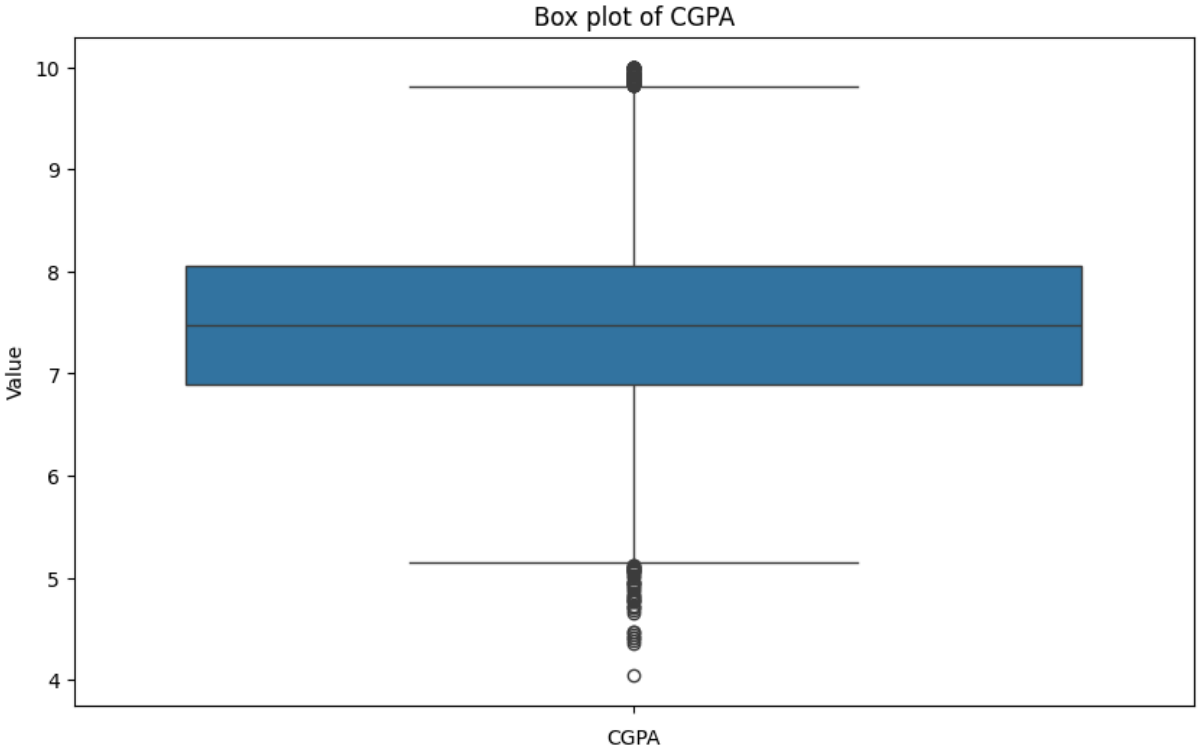
```
Student_ID                       0
Age                              0
Gender                           0
State                            0
Branch                           0
College_Tier                     0
CGPA                             0
Attendance_Percentage            0
DSA_Problems_Solved              0
Aptitude_Score                   0
Communication_Score              0
Projects_Count                   0
Internships_Count                0
Certifications_Count             0
Open_Source_Contributions        0
GitHub_Contributions             0
LinkedIn_Activity_Score          0
Hackathons_Participated          0
Competitive_Programming_Rating   0
Study_Hours_Per_Week             0
Soft_Skills_Score                0
Leadership_Experience            0
Extracurricular_Activities       0
Placement_Status                 0
Package_LPA                      0
dtype: int64
```

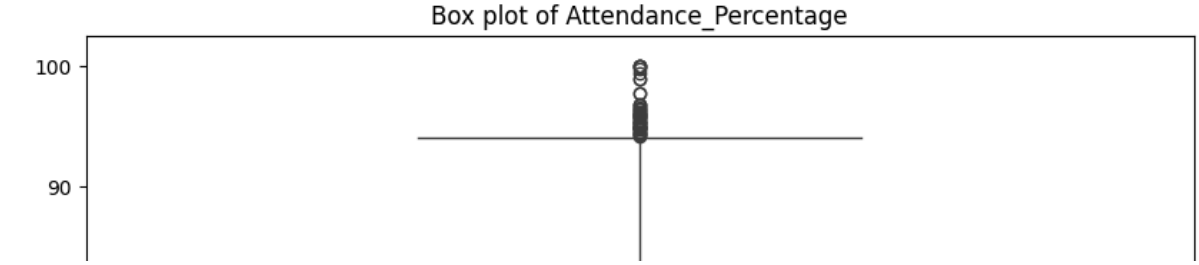
In [7]:
```
numerical_col = df.select_dtypes(include=['float64','int64'])
```

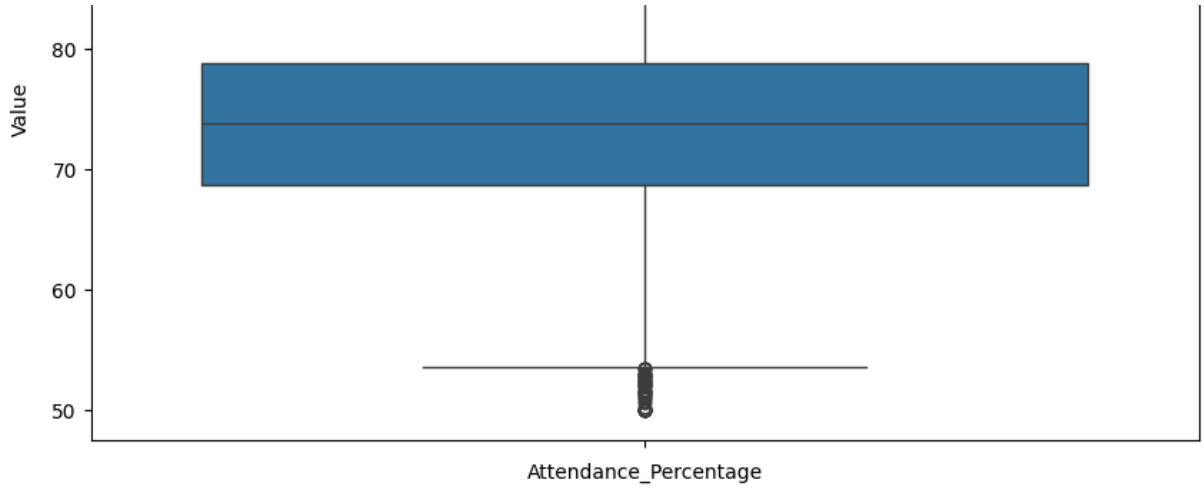
In [8]:
```
for col in numerical_col:
```

```
    plt.figure(figsize=(10, 6))
    sns.boxplot(df[col])
    plt.title(f'Box plot of {col}')
    plt.xlabel(col)
    plt.ylabel('Value')
    plt.show()
```

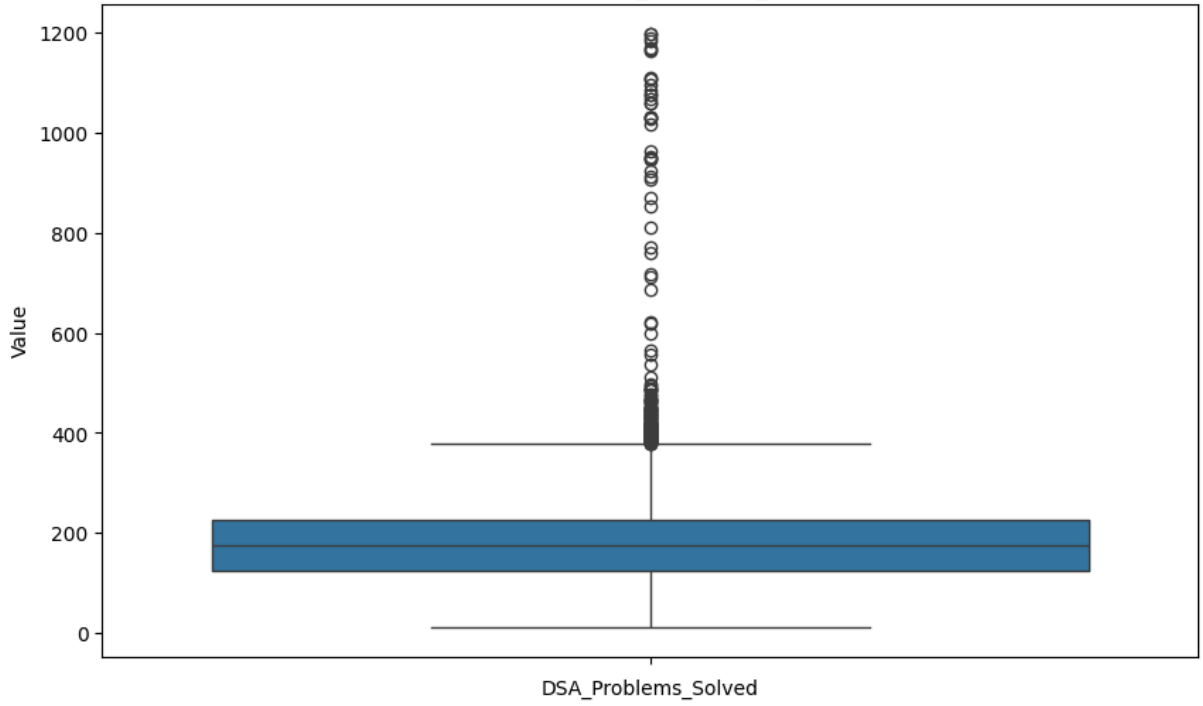




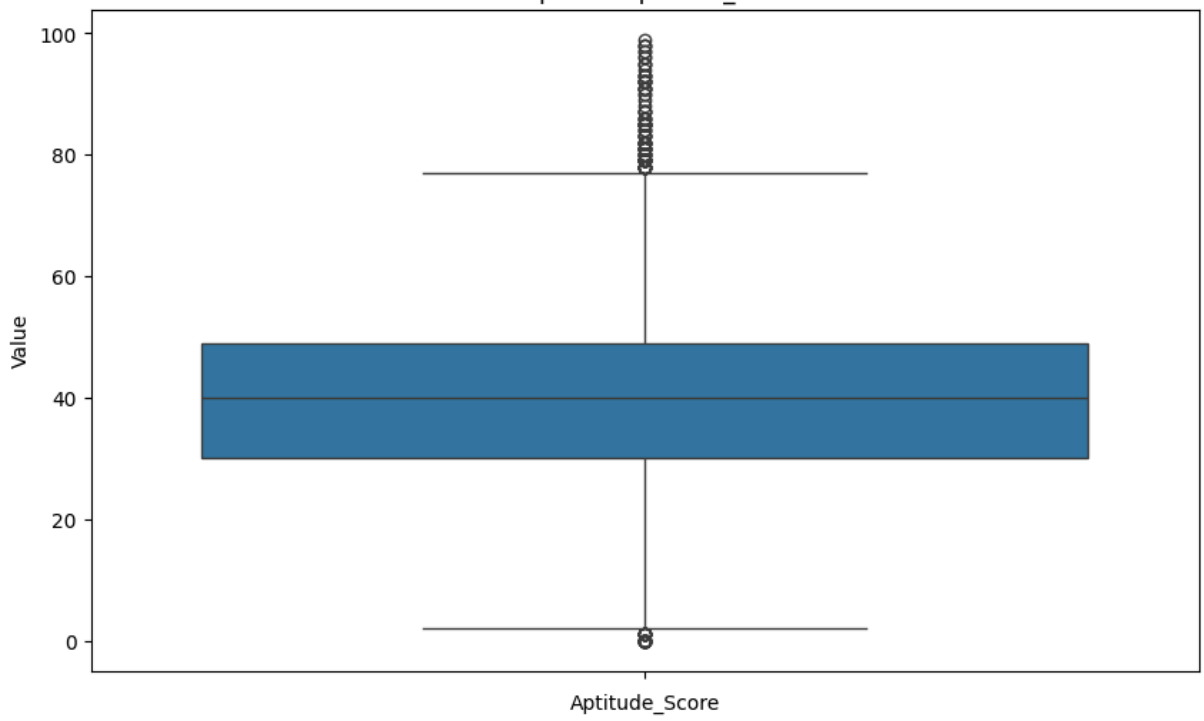




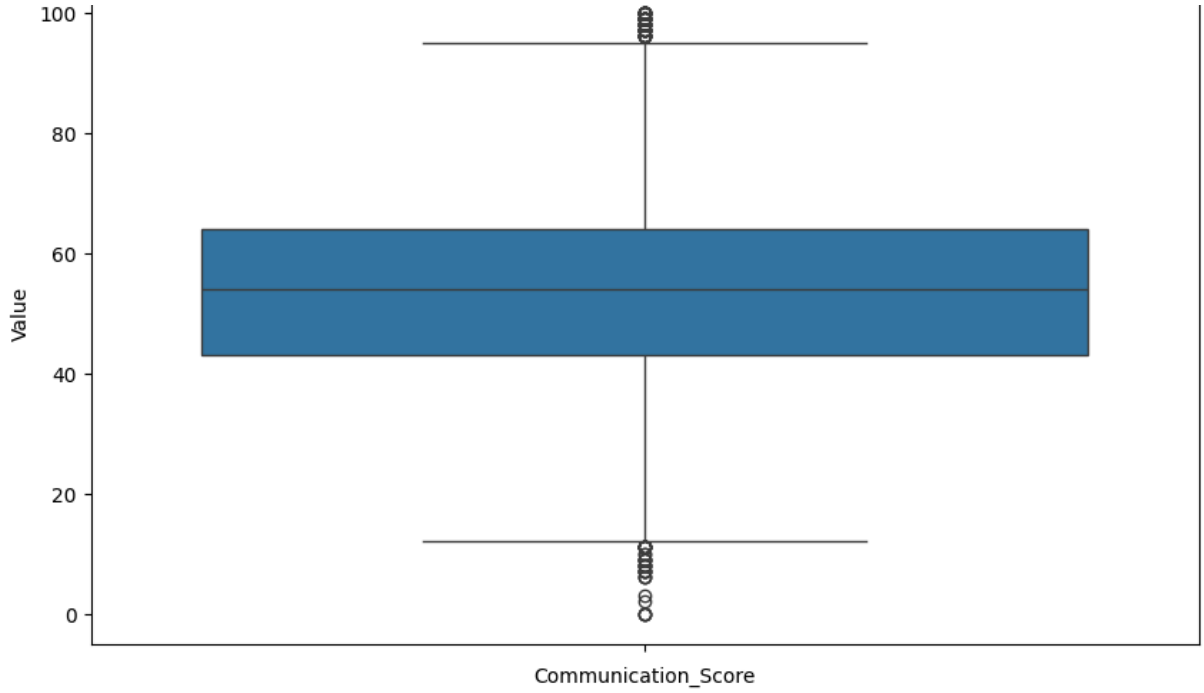
Box plot of DSA_Problems_Solved

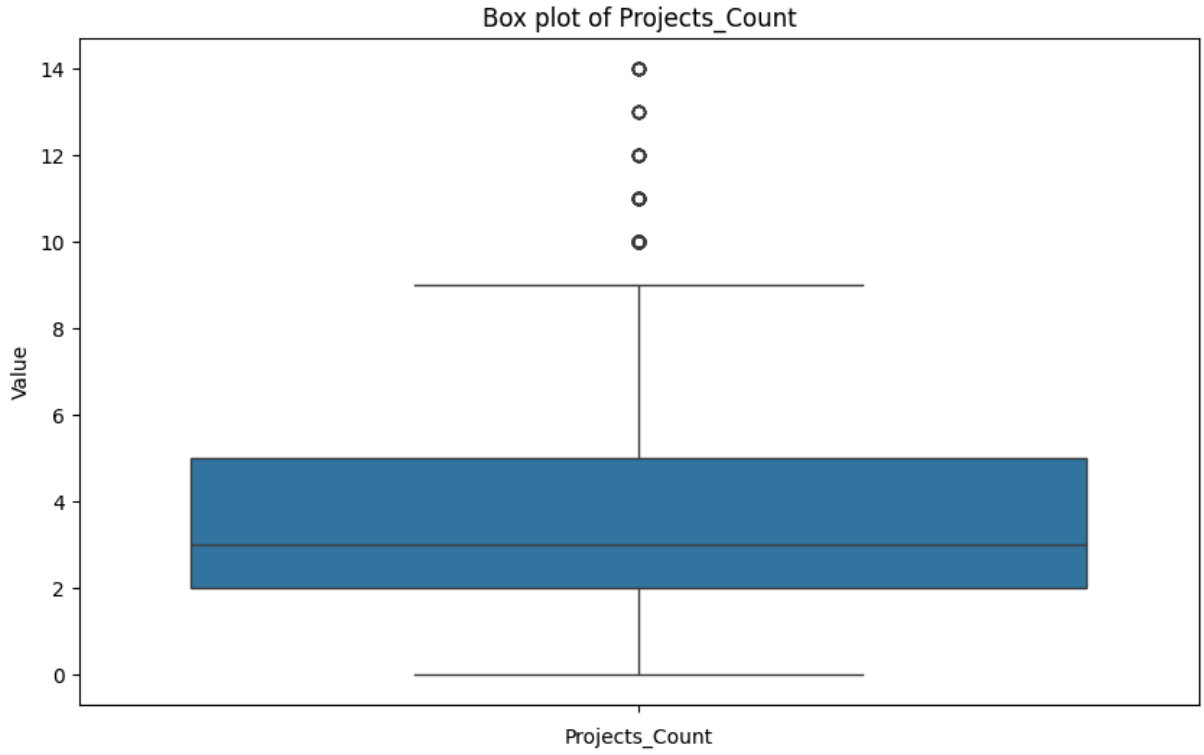


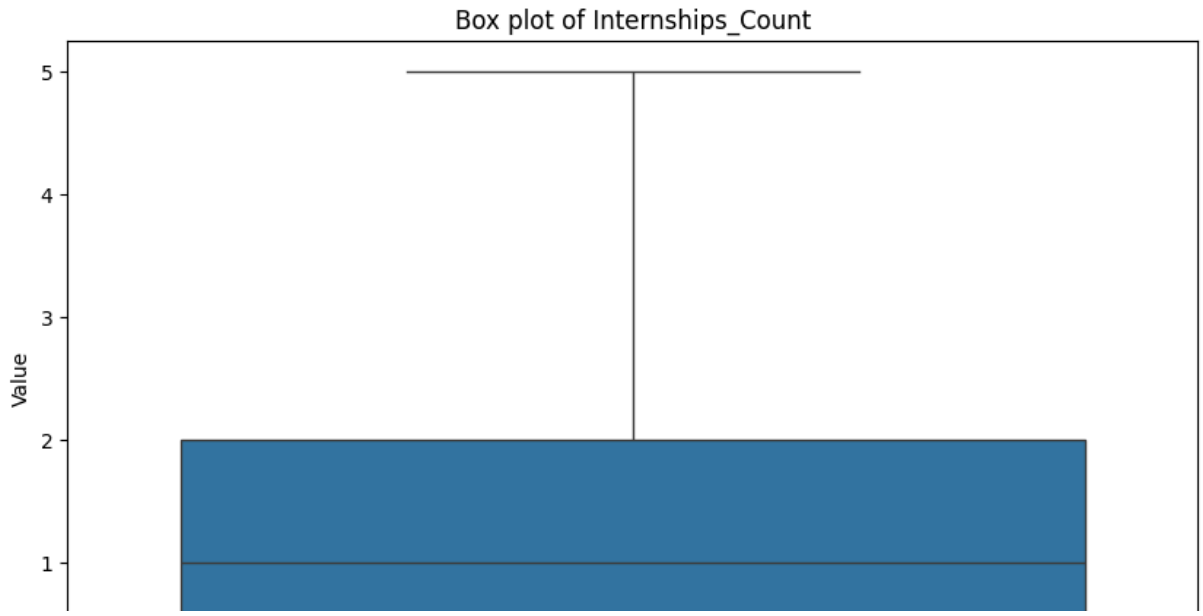
Box plot of Aptitude_Score

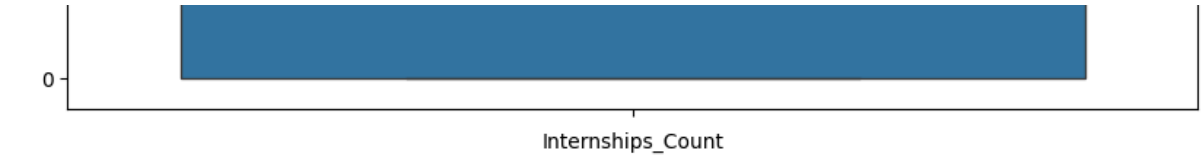


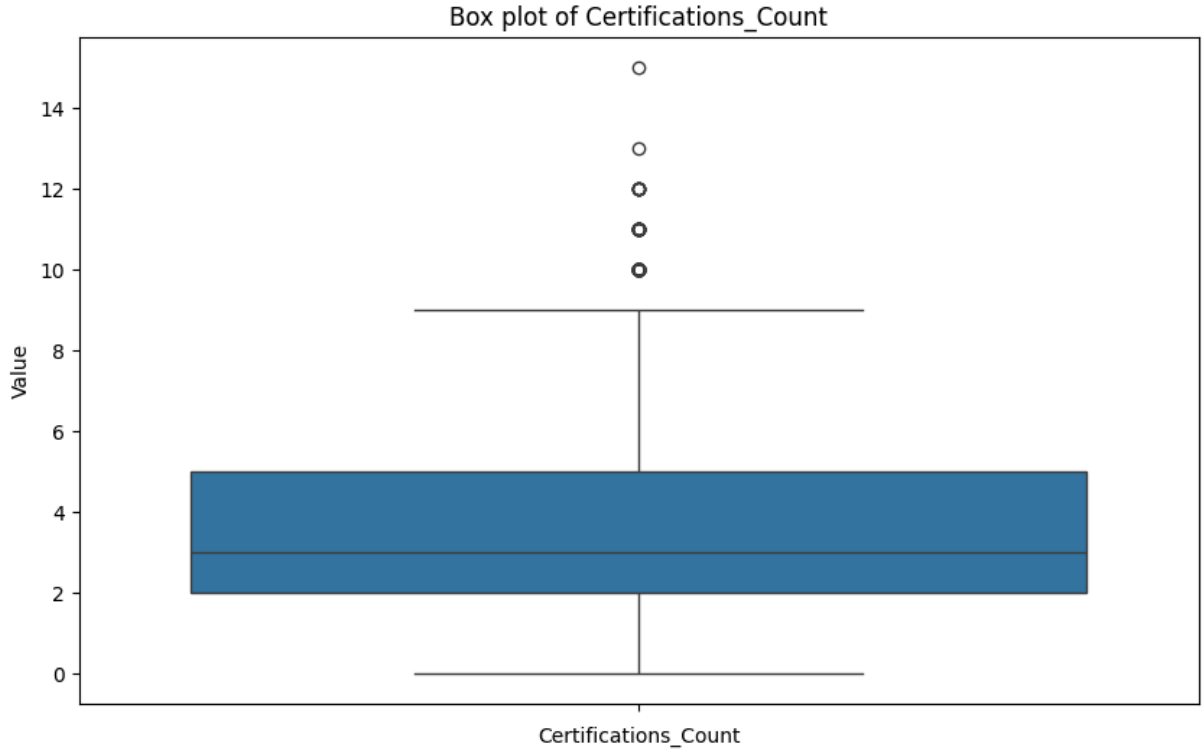
Box plot of Communication_Score

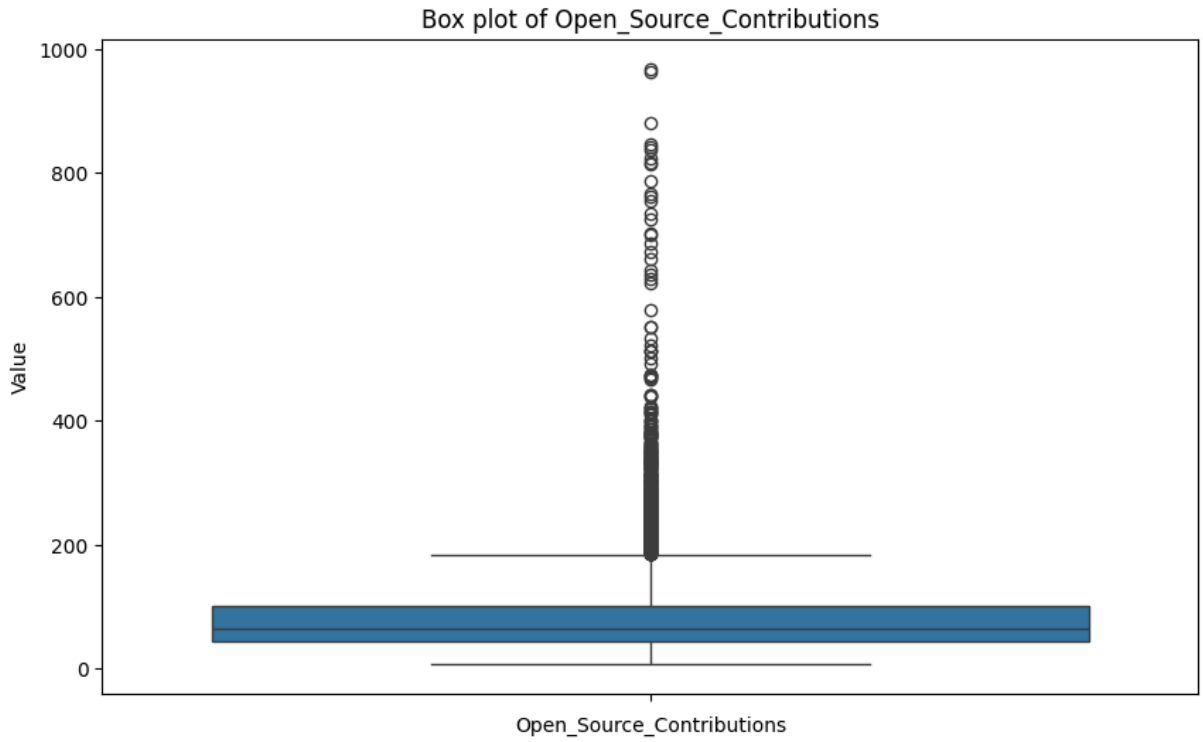

100
80
60
40
20
0
Value
Communication_Score

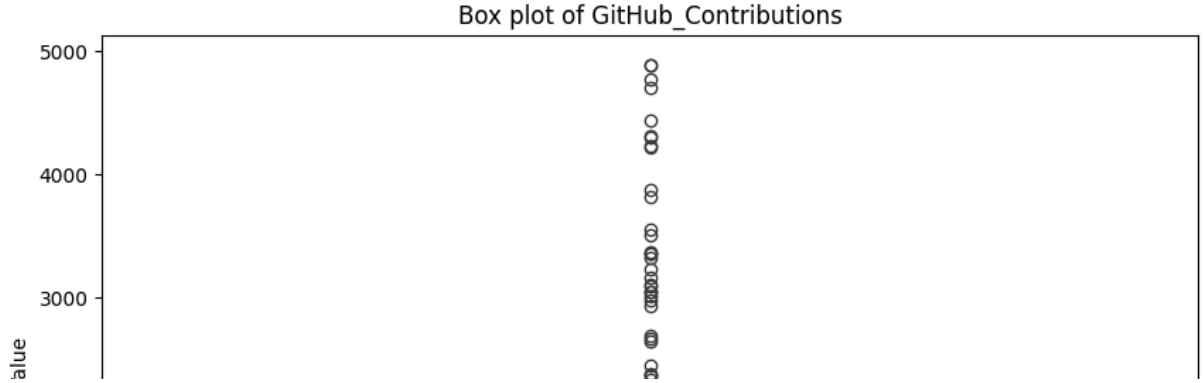

Box plot of Projects_Count
14
12
10
8
6
4
2
0
Value
Projects_Count

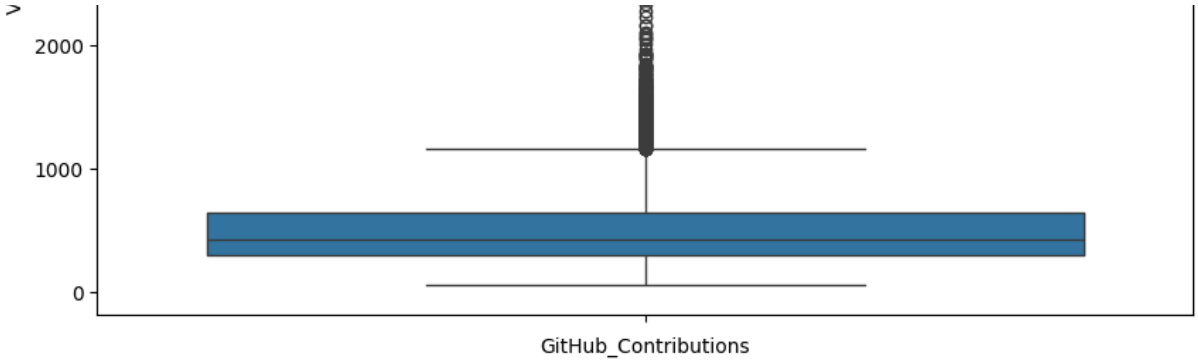

Box plot of Internships_Count
5
4
3
2
1
Value

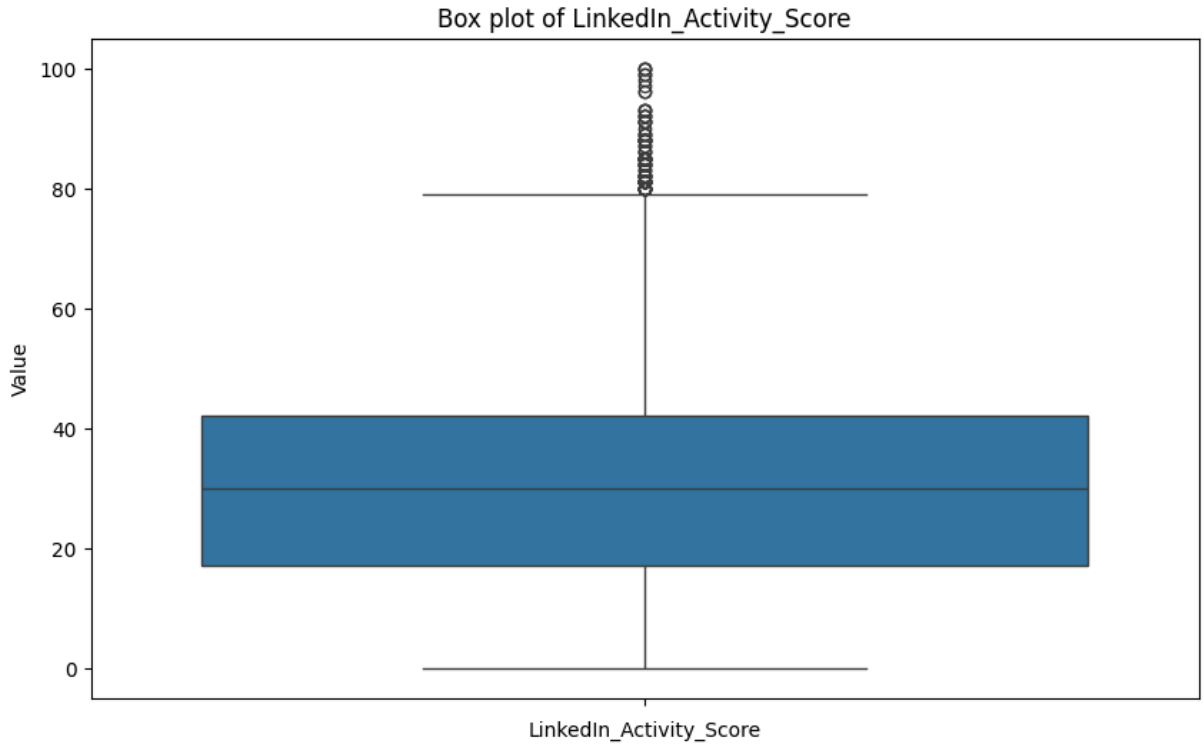

0
Internships_Count

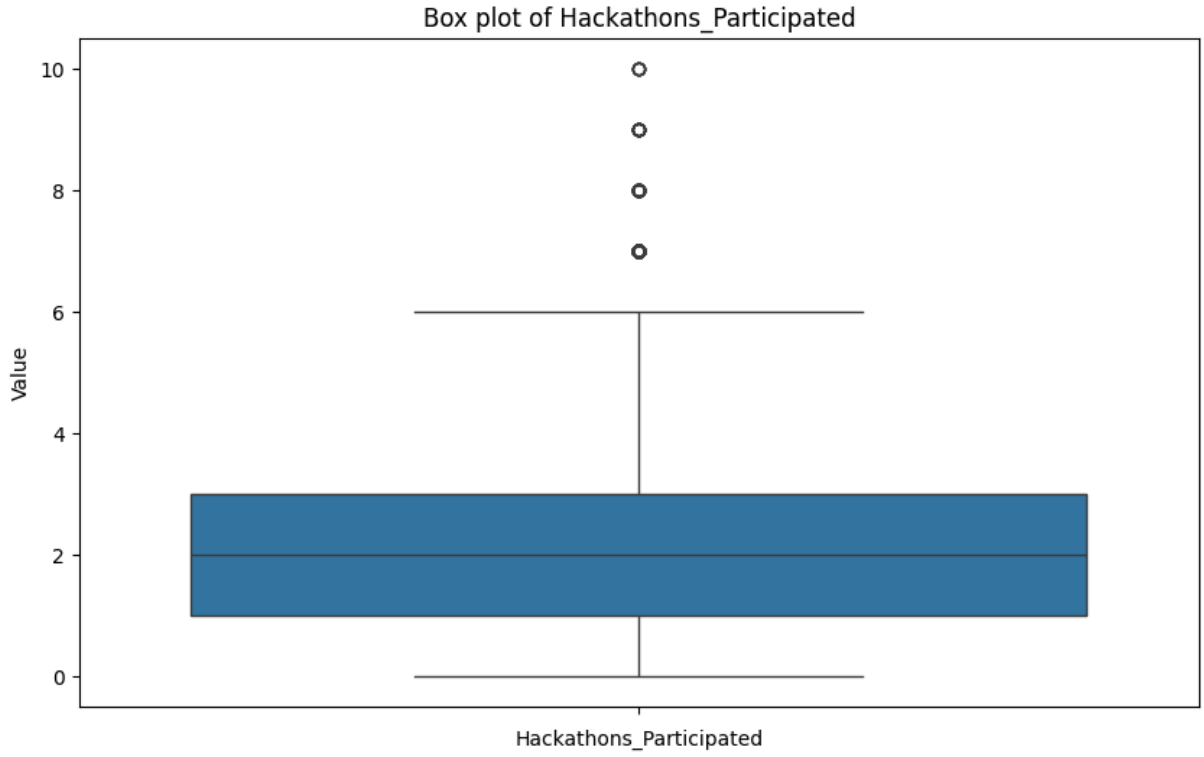

Box plot of Certifications_Count
Value
0
2
4
6
8
10
12
14
Certifications_Count

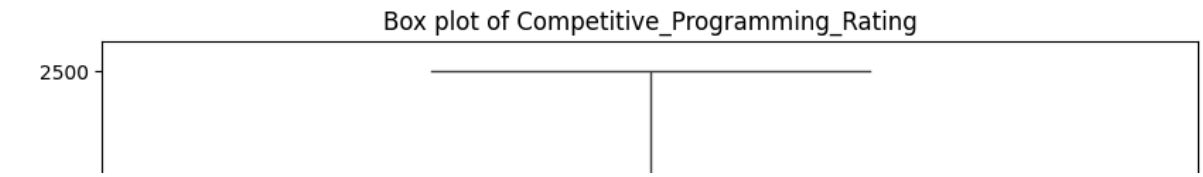

Box plot of Open_Source_Contributions
Value
0
200
400
600
800
1000
Open_Source_Contributions

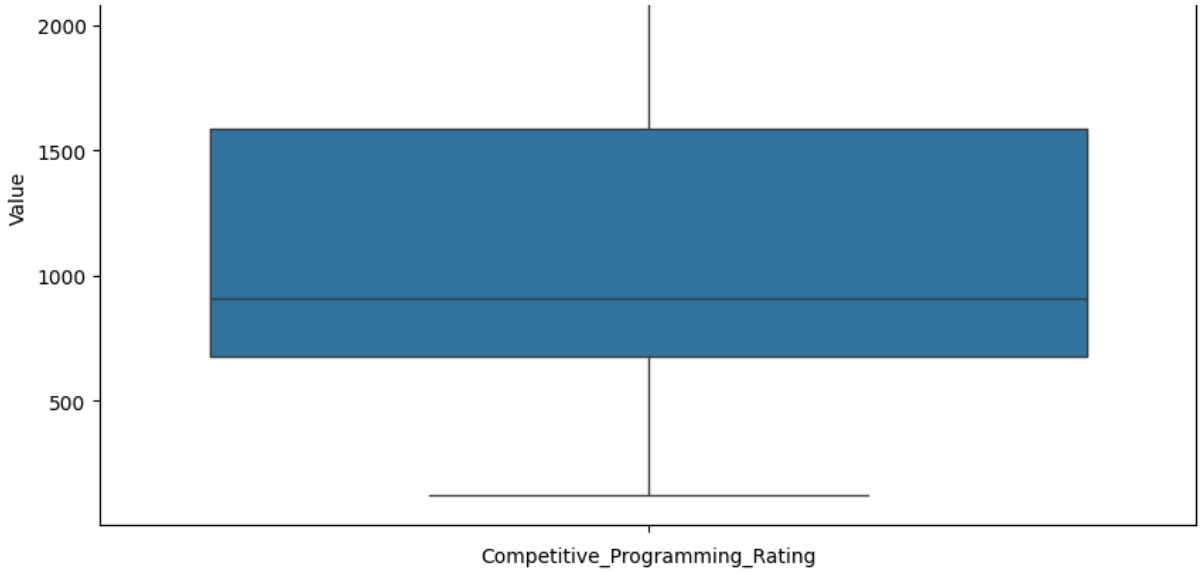

Box plot of GitHub_Contributions
3000
4000
5000

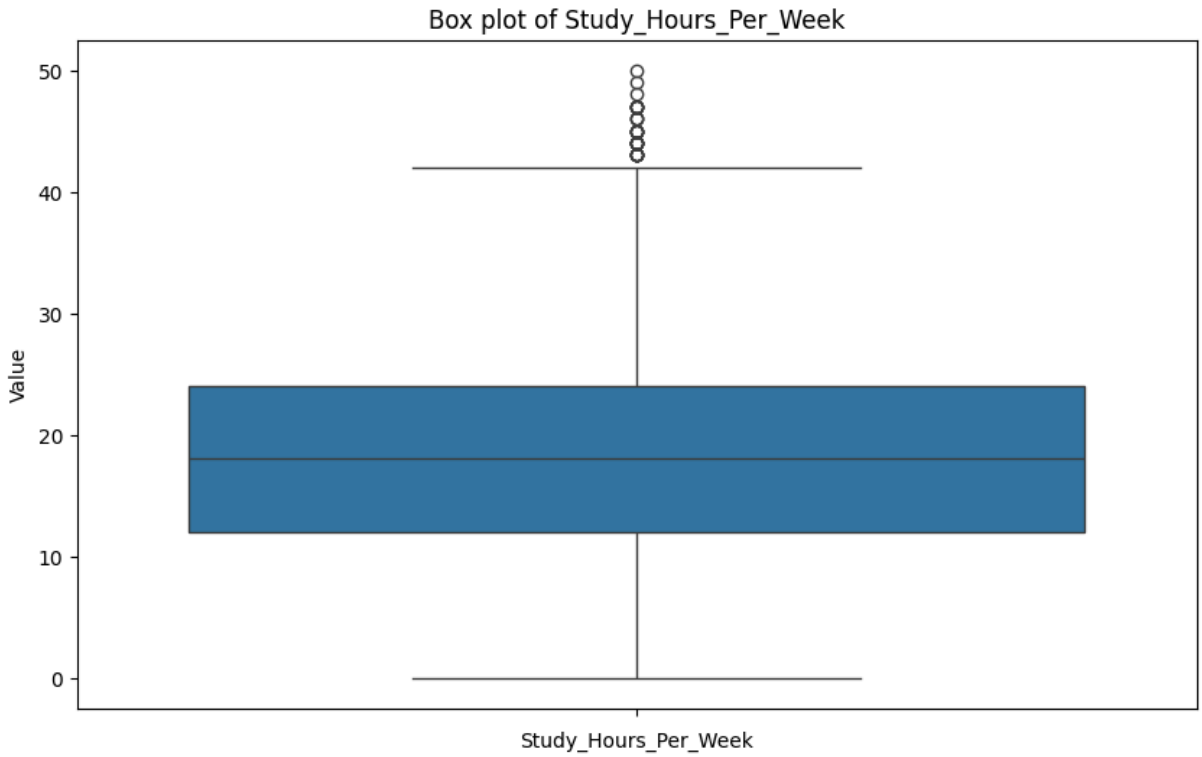

2000
1000
0
GitHub_Contributions


Box plot of LinkedIn_Activity_Score
100
80
60
40
20
0
Value
LinkedIn_Activity_Score


Box plot of Hackathons_Participated
10
8
6
4
2
0
Value
Hackathons_Participated

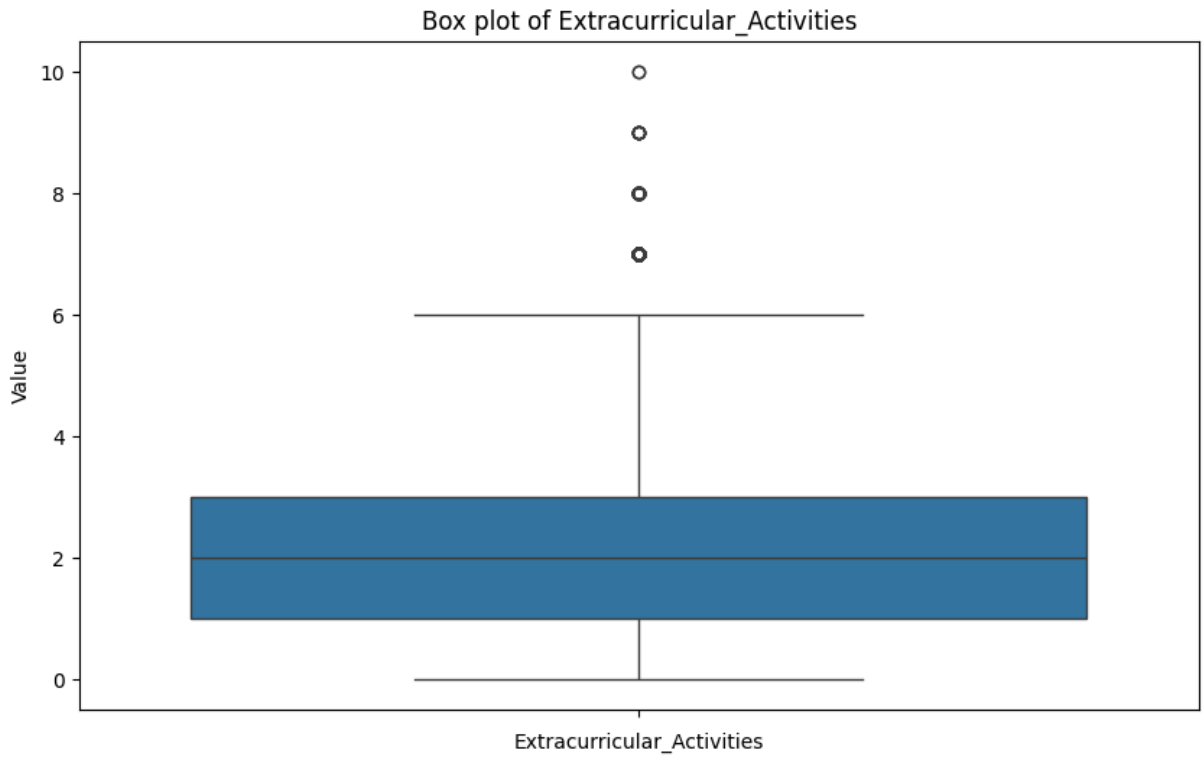

Box plot of Competitive_Programming_Rating
2500

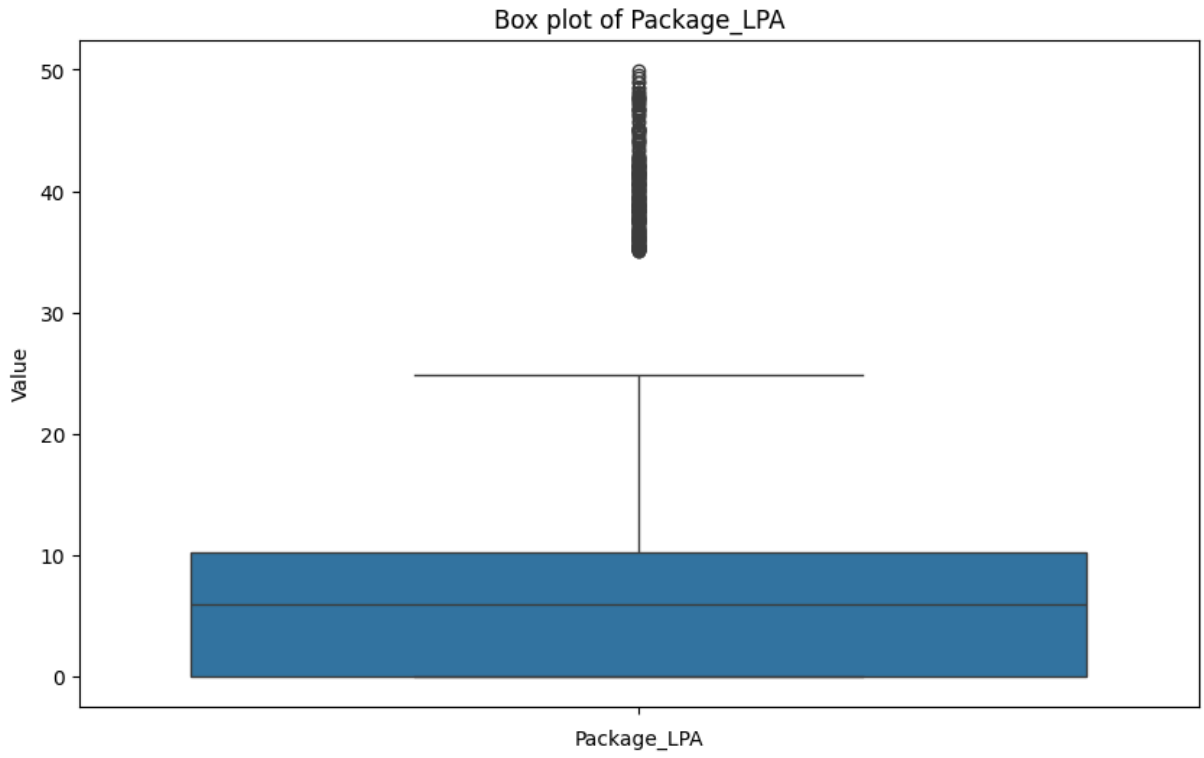

2000
1500
1000
500
Value
Competitive_Programming_Rating


Box plot of Study_Hours_Per_Week
50
40
30
20
10
0
Value
Study_Hours_Per_Week


Box plot of Soft_Skills_Score
90
80
70
60
50
40
30
20
Value

Soft_Skills_Score





In [9]:

```
#handling outliers
q1 = df["Age"].quantile(0.25)
q3 = df["Age"].quantile(0.75)
iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr
#df = df[(df["Age"] >= lower_bound) & (df["Age"] <= upper_bound)]

outliers = df[(df["Age"] < lower_bound) | (df["Age"] > upper_bound)]
print(outliers)

df["Age"] =df["Age"].clip(lower=lower_bound, upper=upper_bound)
```

```
Empty DataFrame
Columns: [Student_ID, Age, Gender, State, Branch, College_Tier, CGPA, Attendance_Pe
rcentage, DSA_Problems_Solved, Aptitude_Score, Communication_Score, Projects_Count,
Internships_Count, Certifications_Count, Open_Source_Contributions, GitHub_Contribu
tions, LinkedIn_Activity_Score, Hackathons_Participated, Competitive_Programming_Ra
ting, Study_Hours_Per_Week, Soft_Skills_Score, Leadership_Experience, Extracurricul
ar_Activities, Placement_Status, Package_LPA]
Index: []

[0 rows x 25 columns]
```

In [10]:
```
df = df.drop("Student_ID", axis=1)
df = df.drop("Placement_Status", axis=1)
```

In [ ]:
```

```

Train test split

Feature Selection and Regression Model

In this section we will:

1. Split the data into training and testing sets.
2. Apply **Variance Threshold** feature selection.
3. Apply **Sequential Feature Selection (SFS)**.
4. Apply **RFE (Recursive Feature Elimination)**.
5. Apply **RFECV (RFE with Cross-Validation)**.
6. Train a simple **Linear Regression** model.
7. Evaluate the model using **R^2, RMSE, MAE and MSE**.

> **Note:** Since this is a regression problem, we use regression scores such as `r2`, not classification `accuracy`.

In [19]:
```
from sklearn.model_selection import train_test_split

X = df.drop("Package_LPA", axis=1)
y = df["Package_LPA"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_s

print("Training data:", X_train.shape)
print("Testing data:", X_test.shape)
```

```
Training data: (12000, 14)
Testing data: (3000, 14)
```

In [18]:
```
# Select only numeric columns
# This keeps the code simple and avoids problems with text/categorical columns.

numeric_data = df.select_dtypes(include=["int64", "float64"]).copy()
```

```
X = numeric_data.drop("Package_LPA", axis=1)
y = numeric_data["Package_LPA"]

print("Features:", list(X.columns))
print("Number of features:", X.shape[1])
```

```
Features: ['Attendance_Percentage', 'Study_Hours_Per_Week', 'Soft_Skills_Score', 'H
ackathons_Participated', 'Aptitude_Score', 'Certifications_Count', 'Communication_S
core', 'Projects_Count', 'Internships_Count', 'CGPA', 'Open_Source_Contributions',
'GitHub_Contributions', 'Competitive_Programming_Rating', 'DSA_Problems_Solved']
Number of features: 14
```

Apply Feature Selection

1. Variance-Based Feature Selection

`VarianceThreshold` removes features whose values do not vary enough.

Here, a threshold of `0.1` is used.

In [20]:

```
from sklearn.feature_selection import VarianceThreshold

variance_selector = VarianceThreshold(threshold=0.1)

X_train_var = variance_selector.fit_transform(X_train)
X_test_var = variance_selector.transform(X_test)

var_features = X_train.columns[variance_selector.get_support()]

print("Variance Selected Features:")
print(list(var_features))
```

```
Variance Selected Features:
['Attendance_Percentage', 'Study_Hours_Per_Week', 'Soft_Skills_Score', 'Hackathons_
Participated', 'Aptitude_Score', 'Certifications_Count', 'Communication_Score', 'Pr
ojects_Count', 'Internships_Count', 'CGPA', 'Open_Source_Contributions', 'GitHub_Co
ntributions', 'Competitive_Programming_Rating', 'DSA_Problems_Solved']
```

Evaluation Function

We will use four regression metrics:

- **R^2** → higher is better
- **RMSE** → lower is better
- **MAE** → lower is better
- **MSE** → lower is better

In [22]:

```
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import numpy as np

def evaluate_model(X_train_selected, X_test_selected, y_train, y_test):
    model = LinearRegression()
    model.fit(X_train_selected, y_train)
```

```
    y_pred = model.predict(X_test_selected)

    r2 = r2_score(y_test, y_pred)
    mse = mean_squared_error(y_test, y_pred)
    rmse = np.sqrt(mse)
    mae = mean_absolute_error(y_test, y_pred)

    return r2, rmse, mae, mse
```

In [23]:

```
# TRAIN AND EVALUATE ALL FEATURE SELECTION METHODS

results = []

# Variance
r2, rmse, mae, mse = evaluate_model(
    X_train[var_features], X_test[var_features], y_train, y_test
)
results.append(["Variance", r2, rmse, mae, mse])
```

In [39]:

```
# Train Linear Regression using Variance Selected Features

from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error, mean_absolute_error
import numpy as np

# Create the model
model_var = LinearRegression()

# Train the model
model_var.fit(X_train_var, y_train)

# Predict on test data
y_pred_var = model_var.predict(X_test_var)

# Calculate metrics
r2_var = r2_score(y_test, y_pred_var)
mse_var = mean_squared_error(y_test, y_pred_var)
rmse_var = np.sqrt(mse_var)
mae_var = mean_absolute_error(y_test, y_pred_var)

# Print results
print("Variance-Based Feature Selection Model")
print("--------------------------------------")
print("R²   :", round(r2_var, 4))
print("RMSE :", round(rmse_var, 4))
print("MAE  :", round(mae_var, 4))
print("MSE  :", round(mse_var, 4))
```

```
Variance-Based Feature Selection Model
--------------------------------------
R²   : 0.5309
RMSE : 4.993
MAE  : 3.5755
MSE  : 24.9304
```

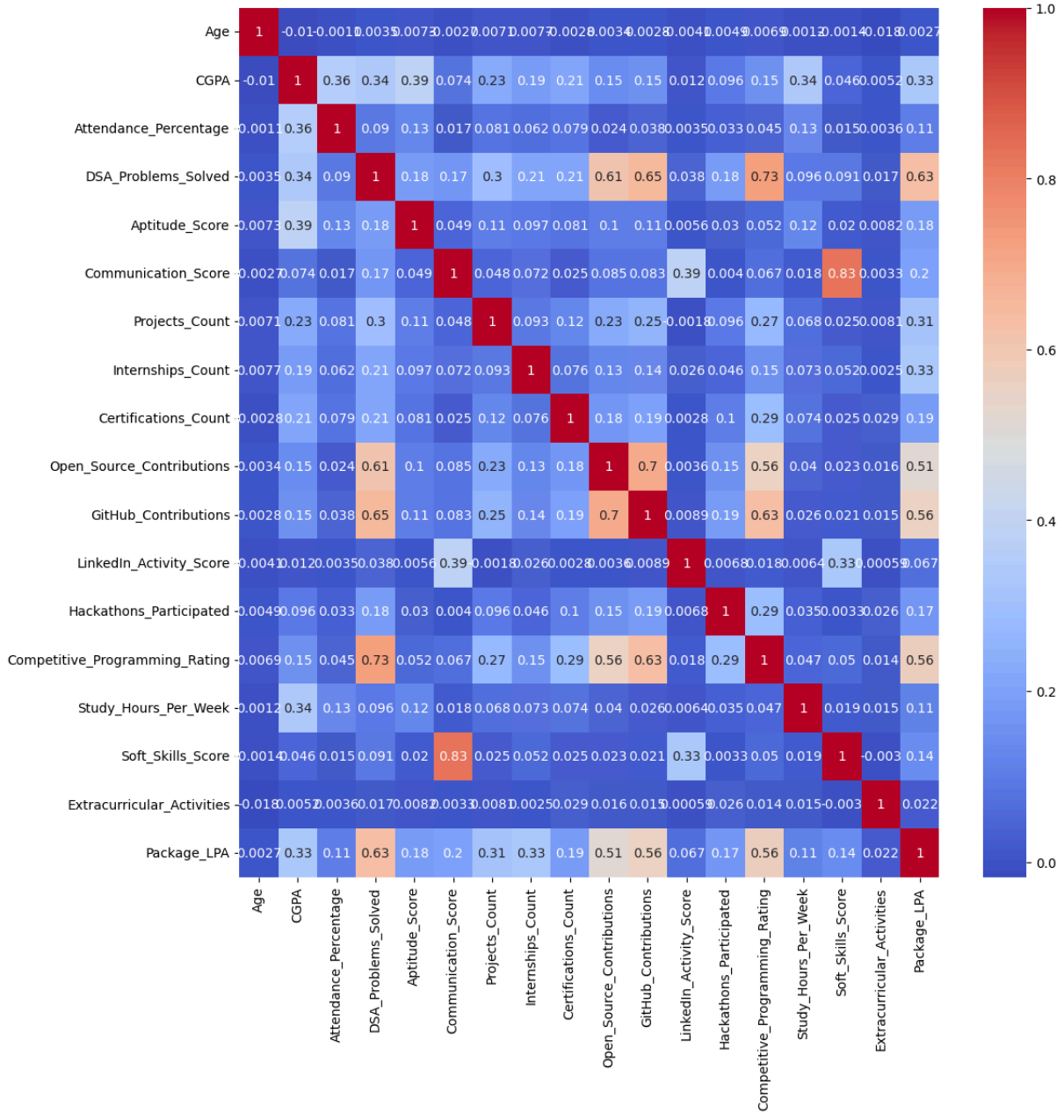
Apply Feature Selection

2. Filter Based

Correlation based

In [12]:
```
corr = df.corr(numeric_only=True)
```

In [13]:
```
fig = plt.figure(figsize=(12, 12))
sns.heatmap(corr, annot=True, cmap="coolwarm")
plt.show()
```



Correlation With Label

In [14]:
```
correlation = df.corr(numeric_only=True)["Package_LPA"].sort_values(ascending=Tru

print(correlation)
```

```
Age                              0.002745
Extracurricular_Activities       0.022488
LinkedIn_Activity_Score          0.067193
Attendance_Percentage            0.107772
Study_Hours_Per_Week             0.109882
Soft_Skills_Score                0.139597
Hackathons_Participated          0.168321
Aptitude_Score                   0.181072
Certifications_Count             0.185551
Communication_Score              0.204367
Projects_Count                   0.309873
Internships_Count                0.326769
CGPA                             0.330195
Open_Source_Contributions        0.511289
GitHub_Contributions             0.561650
Competitive_Programming_Rating   0.564983
DSA_Problems_Solved              0.627133
Package_LPA                      1.000000
Name: Package_LPA, dtype: float64
```

In [15]:

```python
# Keep features with |correlation| >= 0.1
selected_features = correlation[correlation.abs() >= 0.1].index

# Create new dataframe
df = df[selected_features]

print(df.columns)
```

```
Index(['Attendance_Percentage', 'Study_Hours_Per_Week', 'Soft_Skills_Score',
       'Hackathons_Participated', 'Aptitude_Score', 'Certifications_Count',
       'Communication_Score', 'Projects_Count', 'Internships_Count', 'CGPA',
       'Open_Source_Contributions', 'GitHub_Contributions',
       'Competitive_Programming_Rating', 'DSA_Problems_Solved', 'Package_LPA'],
      dtype='object')
```

In [16]:

```python
from sklearn.feature_selection import VarianceThreshold

# Create selector
selector = VarianceThreshold(threshold=0.1)

# Apply to features only
X = df.drop("Package_LPA", axis=1)
y = df["Package_LPA"]

X_selected = selector.fit_transform(X)

# Get selected feature names
selected_features = X.columns[selector.get_support()]

# Create new dataframe
X = pd.DataFrame(X_selected, columns=selected_features, index=df.index)

print("Selected features:")
print(selected_features)
```

```
Selected features:
Index(['Attendance_Percentage', 'Study_Hours_Per_Week', 'Soft_Skills_Score',
       'Hackathons_Participated', 'Aptitude_Score', 'Certifications_Count',
       'Communication_Score', 'Projects_Count', 'Internships_Count', 'CGPA',
       'Open_Source_Contributions', 'GitHub_Contributions',
       'Competitive_Programming_Rating', 'DSA_Problems_Solved'],
      dtype='object')
```

Sequential Feature Selection

In [35]:
```
from sklearn.feature_selection import SequentialFeatureSelector
from sklearn.linear_model import LinearRegression

model = LinearRegression()

# Select up to 5 features, or all features if there are fewer than 5
n_select = min(5, X_train.shape[1])

sfs = SequentialFeatureSelector(
    model,
    n_features_to_select=n_select,
    direction="forward",
    scoring="r2",
    cv=5
)

sfs.fit(X_train, y_train)

sfs_features = X_train.columns[sfs.get_support()]

print("SFS Selected Features:")
print(list(sfs_features))

# Evaluate the SFS model immediately after feature selection
r2_sfs, rmse_sfs, mae_sfs, mse_sfs = evaluate_model(
    X_train[sfs_features], X_test[sfs_features], y_train, y_test
)

print("\n===== SFS Model Metrics =====")
print("R²   :", round(r2_sfs, 4))
print("RMSE :", round(rmse_sfs, 4))
print("MAE  :", round(mae_sfs, 4))
print("MSE  :", round(mse_sfs, 4))
```

```
SFS Selected Features:
['Internships_Count', 'CGPA', 'GitHub_Contributions', 'Competitive_Programming_Rati
ng', 'DSA_Problems_Solved']

===== SFS Model Metrics =====
R²   : 0.5129
RMSE : 5.0878
MAE  : 3.6339
MSE  : 25.8861
```

In [31]:
```
# Train model using SFS selected features

model_sfs = LinearRegression()
model_sfs.fit(X_train[sfs_features], y_train)

# Prediction
y_pred_sfs = model_sfs.predict(X_test[sfs_features])

# Metrics
r2_sfs = r2_score(y_test, y_pred_sfs)
mse_sfs = mean_squared_error(y_test, y_pred_sfs)
rmse_sfs = np.sqrt(mse_sfs)
mae_sfs = mean_absolute_error(y_test, y_pred_sfs)
```

```
print("===== SFS Model =====")
print("R²   :", round(r2_sfs, 4))
print("RMSE :", round(rmse_sfs, 4))
print("MAE  :", round(mae_sfs, 4))
print("MSE  :", round(mse_sfs, 4))
```

```
===== SFS Model =====
R²   : 0.5129
RMSE : 5.0878
MAE  : 3.6339
MSE  : 25.8861
```

RFE — Recursive Feature Elimination

RFE starts with all features and removes less useful features step-by-step.

In [26]:

```
from sklearn.feature_selection import RFE

n_rfe = min(5, X_train.shape[1])

rfe = RFE(
    estimator=LinearRegression(),
    n_features_to_select=n_rfe
)

rfe.fit(X_train, y_train)

rfe_features = X_train.columns[rfe.support_]

print("RFE Selected Features:")
print(list(rfe_features))
```

```
RFE Selected Features:
['Hackathons_Participated', 'Certifications_Count', 'Projects_Count', 'Internships_
Count', 'CGPA']
```

In [32]:

```
# Train model using RFE selected features

model_rfe = LinearRegression()
model_rfe.fit(X_train[rfe_features], y_train)

# Prediction
y_pred_rfe = model_rfe.predict(X_test[rfe_features])

# Metrics
r2_rfe = r2_score(y_test, y_pred_rfe)
mse_rfe = mean_squared_error(y_test, y_pred_rfe)
rmse_rfe = np.sqrt(mse_rfe)
mae_rfe = mean_absolute_error(y_test, y_pred_rfe)

print("===== RFE Model =====")
print("R²   :", round(r2_rfe, 4))
print("RMSE :", round(rmse_rfe, 4))
print("MAE  :", round(mae_rfe, 4))
print("MSE  :", round(mse_rfe, 4))
```

```
===== RFE Model =====
R²   : 0.2662
```

```
RMSE : 6.2446
MAE  : 4.4431
MSE  : 38.9954
```

RFECV — RFE with Cross-Validation

RFECV automatically finds a suitable number of features using cross-validation.

In [27]:

```
from sklearn.feature_selection import RFECV

rfecv = RFECV(
    estimator=LinearRegression(),
    step=1,
    cv=5,
    scoring="r2",
    min_features_to_select=1
)

rfecv.fit(X_train, y_train)

rfecv_features = X_train.columns[rfecv.support_]

print("RFECV Selected Features:")
print(list(rfecv_features))
print("Number of selected features:", len(rfecv_features))
```

```
RFECV Selected Features:
['Attendance_Percentage', 'Study_Hours_Per_Week', 'Soft_Skills_Score', 'Hackathons_
Participated', 'Aptitude_Score', 'Certifications_Count', 'Communication_Score', 'Pr
ojects_Count', 'Internships_Count', 'CGPA', 'Open_Source_Contributions', 'GitHub_Co
ntributions', 'Competitive_Programming_Rating', 'DSA_Problems_Solved']
Number of selected features: 14
```

In [33]:

```
# Train model using RFECV selected features

model_rfecv = LinearRegression()
model_rfecv.fit(X_train[rfecv_features], y_train)

# Prediction
y_pred_rfecv = model_rfecv.predict(X_test[rfecv_features])

# Metrics
r2_rfecv = r2_score(y_test, y_pred_rfecv)
mse_rfecv = mean_squared_error(y_test, y_pred_rfecv)
rmse_rfecv = np.sqrt(mse_rfecv)
mae_rfecv = mean_absolute_error(y_test, y_pred_rfecv)

print("===== RFECV Model =====")
print("R²   :", round(r2_rfecv, 4))
print("RMSE :", round(rmse_rfecv, 4))
print("MAE  :", round(mae_rfecv, 4))
print("MSE  :", round(mse_rfecv, 4))
```

```
===== RFECV Model =====
R²   : 0.5309
RMSE : 4.993
MAE  : 3.5755
MSE  : 24.9304
```

Train and Evaluate All Feature Selection Methods

The same Linear Regression model is trained after each feature-selection method.

In [41]:
```
results = []

# Variance
r2, rmse, mae, mse = evaluate_model(
    X_train[var_features], X_test[var_features], y_train, y_test
)
results.append(["Variance", r2, rmse, mae, mse])

# SFS
r2, rmse, mae, mse = evaluate_model(
    X_train[sfs_features], X_test[sfs_features], y_train, y_test
)
results.append(["SFS", r2, rmse, mae, mse])

# RFE
r2, rmse, mae, mse = evaluate_model(
    X_train[rfe_features], X_test[rfe_features], y_train, y_test
)
results.append(["RFE", r2, rmse, mae, mse])

# RFECV
r2, rmse, mae, mse = evaluate_model(
    X_train[rfecv_features], X_test[rfecv_features], y_train, y_test
)
results.append(["RFECV", r2, rmse, mae, mse])

results_df = pd.DataFrame(
    results,
    columns=["Method", "R2", "RMSE", "MAE", "MSE"]
)

results_df
```

Out[41]:

	Method	R2	RMSE	MAE	MSE
0	Variance	0.530851	4.993032	3.575522	24.930366
1	SFS	0.512866	5.087842	3.633885	25.886133
2	RFE	0.266170	6.244632	4.443125	38.995424
3	RFECV	0.530851	4.993032	3.575522	24.930366

Final Model using RFECV Features

RFECV selected features are used for the final simple Linear Regression model.

In [29]:
```
final_model = LinearRegression()

final_model.fit(X_train[rfecv_features], y_train)

y_pred = final_model.predict(X_test[rfecv_features])
```

```
print("Final Selected Features:")
print(list(rfecv_features))

print("\nFinal Model Metrics")
print("R²   :", round(r2_score(y_test, y_pred), 4))
print("RMSE :", round(np.sqrt(mean_squared_error(y_test, y_pred)), 4))
print("MAE  :", round(mean_absolute_error(y_test, y_pred), 4))
print("MSE  :", round(mean_squared_error(y_test, y_pred), 4))
```

```
Final Selected Features:
['Attendance_Percentage', 'Study_Hours_Per_Week', 'Soft_Skills_Score', 'Hackathons_
Participated', 'Aptitude_Score', 'Certifications_Count', 'Communication_Score', 'Pr
ojects_Count', 'Internships_Count', 'CGPA', 'Open_Source_Contributions', 'GitHub_Co
ntributions', 'Competitive_Programming_Rating', 'DSA_Problems_Solved']

Final Model Metrics
R²   : 0.5309
RMSE : 4.993
MAE  : 3.5755
MSE  : 24.9304
```

Actual vs Predicted Values

The graph compares the actual package values with the values predicted by the final model.

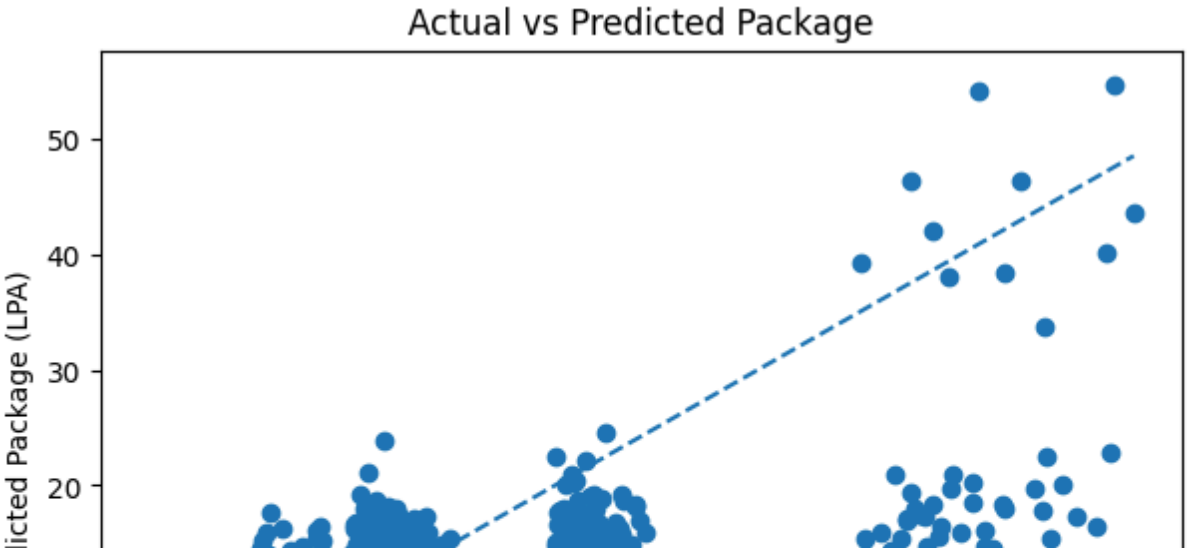
In [30]:

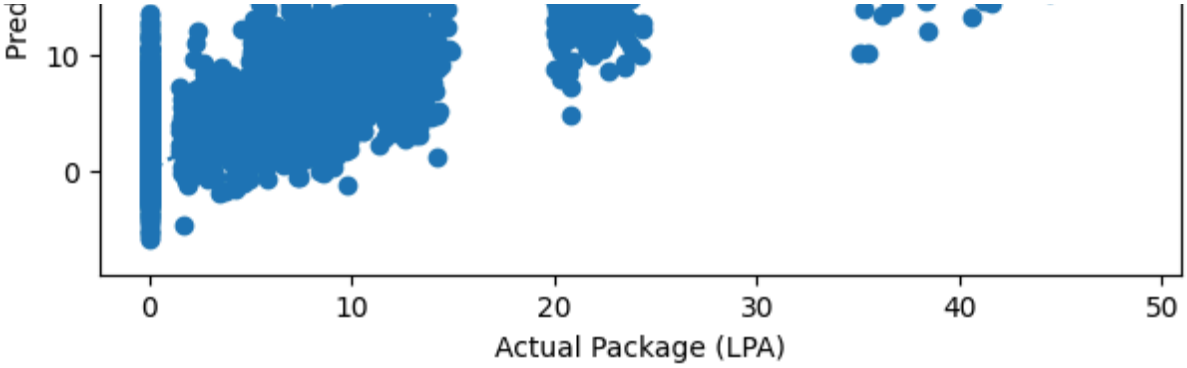
```
plt.figure(figsize=(7, 5))
plt.scatter(y_test, y_pred)

plt.xlabel("Actual Package (LPA)")
plt.ylabel("Predicted Package (LPA)")
plt.title("Actual vs Predicted Package")

# Reference line
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    linestyle="--"
)

plt.show()
```





Conclusion

Feature selection helps remove less useful features before training the regression model.

The model is evaluated using **R^2, RMSE, MAE and MSE**.
The final Linear Regression model uses the features selected by **RFECV**.

```
Dataset
   ↓
Train-Test Split
   ↓
 ┌──────────┬──────────┬──────────┬──────────┐
 ↓          ↓          ↓          ↓
Variance    SFS        RFE        RFECV
 ↓          ↓          ↓          ↓
Linear      Linear     Linear     Linear
Regression  Regression Regression Regression
 ↓          ↓          ↓          ↓
R² RMSE     R² RMSE    R² RMSE    R² RMSE
MAE MSE     MAE MSE    MAE MSE    MAE MSE

R² → HIGH is good
RMSE → LOW is good
MAE → LOW is good
MSE → LOW is good
```

🧠 Feature Selection Methods

Method	Easy Way to Remember	What It Does
Variance	📉 Remove low variation	Removes features with very low variance
SFS	➕ Add features	Adds useful features step-by-step
RFE	❌ Remove features	Removes less important features
RFECV	🔄 RFE + CV	RFE + Cross-Validation automatically chooses features